Practical 1

Part A — Practice Programs

1. Vending machine

CODE:

```
import java.util.Scanner;
public class VendingMachine
{
    enum Coin
    {
        ONE, TWO, FIVE, TEN
    }
    public static void main(String[] args)
    {
        int snackPrice = 15;
        int totalAmount = 0;
        int coinValue = 0;
        Scanner sc = new Scanner(System.in);

        while (totalAmount<snackPrice)
        {
            System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");
            String input = sc.next();
            Coin coin = null ;
            switch (input)
```

```
{
    case "ONE":
        coin = Coin.ONE;
        break;
    case "TWO":
        coin = Coin.TWO;
        break;
    case "FIVE":
        coin = Coin.FIVE;
        break;
    case "TEN":
        coin = Coin.TEN;
        break;
    default:
        System.out.println("Invalid Coin!");
        continue;
}
switch (coin)
{
    case ONE:
        coinValue=1;
        break;
    case TWO:
        coinValue=2;
        break;
```

```
                case FIVE:
                    coinValue=5;
                    break;
                case TEN:
                    coinValue=10;
                    break;
            }
            totalAmount += coinValue;
            System.out.println("Total paid: " + totalAmount);
        }
        System.out.println("Paid. Change: " + (totalAmount-
snackPrice));
        sc.close();
    }
}
```

OUTPUT:

```
PS E:\java\25aiml044> java VendingMachine.java
Enter coin (ONE, TWO, FIVE, TEN): FIVE
Total paid: 5
Enter coin (ONE, TWO, FIVE, TEN): ONE
Total paid: 6
Enter coin (ONE, TWO, FIVE, TEN): TWO
Total paid: 8
Enter coin (ONE, TWO, FIVE, TEN): TEN
Total paid: 18
Paid. Change: 3
```

2. **Toll booth**

CODE:

```
import java.util.Scanner;
public class TollBooth
{
    record Vehicle(String number, String type){}
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int totalToll=0;
        int bikeCount=0;
        int carCount=0;
        int truckCount=0;
        while(true)
        {
            System.out.print("Enter vehicle number (or done to stop): ");
            String number = sc.next();
            if (number.equals("done"))
            {
                break;
            }
            System.out.print("Enter vehicle type (bike/car/truck): ");
            String type = sc.next();
            Vehicle vehicle = new Vehicle(number, type);
            switch (vehicle.type())
            {
                case "bike":
```

```
                totalToll += 20;
                bikeCount++;
                break;
            case "car":
                totalToll += 50;
                carCount++;
                break;
            case "truck":
                totalToll += 150;
                truckCount++;
                break;
            default:
                System.out.println("Invalid vehicle type!");
        }
    }
    System.out.println("Total toll: " + totalToll);
    if(bikeCount >= carCount && bikeCount >= truckCount)
    {
        System.out.println("Most frequent: bike");
    }
    else if(carCount >= bikeCount && carCount >= truckCount)
    {
        System.out.println("Most frequent: car");
    }
    else
```

```
        {
            System.out.println("Most frequent: truck");
        }
        sc.close();
    }
}
```

OUTPUT:

```
PS E:\java\25aiml044> java TollBooth.java
Enter vehicle number (or done to stop): 0000
Enter vehicle type (bike/car/truck): bike
Enter vehicle number (or done to stop): 0007
Enter vehicle type (bike/car/truck): car
Enter vehicle number (or done to stop): 1010
Enter vehicle type (bike/car/truck): truck
Enter vehicle number (or done to stop): done
Total toll: 220
Most frequent: bike
```

3. Rock-Paper-Scissors-Lizard-Spock (RPSLS)

CODE:

```
import java.util.Random;
import java.util.Scanner;

public class RPSLS
{

    enum Move
```

```
{
    ROCK, PAPER, SCISSORS, LIZARD, SPOCK
}

static int winner(Move a, Move b)
{

    if (a == b)
        return 0;

    switch (a)
    {

        case ROCK:
            if (b == Move.SCISSORS || b == Move.LIZARD)
                return 1;
            else
                return -1;

        case PAPER:
            if (b == Move.ROCK || b == Move.SPOCK)
                return 1;
            else
                return -1;
```

```
            case SCISSORS:
                if (b == Move.PAPER || b == Move.LIZARD)
                    return 1;
                else
                    return -1;

            case LIZARD:
                if (b == Move.PAPER || b == Move.SPOCK)
                    return 1;
                else
                    return -1;

            case SPOCK:
                if (b == Move.ROCK || b == Move.SCISSORS)
                    return 1;
                else
                    return -1;
        }

        return 0;
    }

    public static void main(String[] args)
    {
```

```
Scanner sc = new Scanner(System.in);
Random random = new Random();

int playerScore = 0, computerScore = 0;

for (int i = 1; i <= 5; i++)
{

   System.out.println("\n---Round " + i + "---");
   System.out.println("1. ROCK");
   System.out.println("2. PAPER");
   System.out.println("3. SCISSORS");
   System.out.println("4. LIZARD");
   System.out.println("5. SPOCK");
   System.out.print("Enter your choice: ");

   int choice = sc.nextInt();

   Move player = null;

   switch (choice)

   {
      case 1:
         player = Move.ROCK;
```

```
            break;
        case 2:
            player = Move.PAPER;
            break;
        case 3:
            player = Move.SCISSORS;
            break;
        case 4:
            player = Move.LIZARD;
            break;
        case 5:
            player = Move.SPOCK;
            break;
        default:
            System.out.println("Invalid choice");
            i--;
            continue;
    }

    int computerChoice = random.nextInt(5) + 1;
    Move computer = null;

    switch (computerChoice)
    {
        case 1:
```

```
            computer = Move.ROCK;
            break;
        case 2:
            computer = Move.PAPER;
            break;
        case 3:
            computer = Move.SCISSORS;
            break;
        case 4:
            computer = Move.LIZARD;
            break;
        case 5:
            computer = Move.SPOCK;
            break;
    }

    System.out.println("Your Move: " + player);
    System.out.println("Computer Move: " + computer);

    int result = winner(player, computer);

    if (result == 1)
    {
        System.out.println("You win this round.");
        playerScore++;
```

```
            }
            else if (result == -1)
            {
                System.out.println("Computer wins this round.");
                computerScore++;
            }
            else
            {
                System.out.println("Round Tie.");
            }
        }

        System.out.println("\nFinal Score");
        System.out.println("You: " + playerScore);
        System.out.println("Computer: " + computerScore);

        if (playerScore > computerScore)
            System.out.println("You win " + playerScore + "-" +
computerScore);
        else if (computerScore > playerScore)
            System.out.println("Computer wins " + computerScore +
"-" + playerScore);
        else
            System.out.println("Match Tie " + playerScore + "-" +
computerScore);
sc.close();}
```

}

OUTPUT:

```
PS E:\java\25aiml044> java RPSLS.java
---Round 1---
1. ROCK
2. PAPER
3. SCISSORS
4. LIZARD
5. SPOCK
Enter your choice: 1
Your Move: ROCK
Computer Move: SPOCK
Computer wins this round.

---Round 2---
1. ROCK
2. PAPER
3. SCISSORS
4. LIZARD
5. SPOCK
Enter your choice: 2
Your Move: PAPER
Computer Move: LIZARD
Computer wins this round.

---Round 3---
1. ROCK
2. PAPER
3. SCISSORS
4. LIZARD
5. SPOCK
Enter your choice: 3
Your Move: SCISSORS
Computer Move: SPOCK
Computer wins this round.
---Round 4---
1. ROCK
2. PAPER
3. SCISSORS
4. LIZARD
5. SPOCK
Enter your choice: 4
Your Move: LIZARD
Computer Move: ROCK
Computer wins this round.

---Round 5---
1. ROCK
2. PAPER
3. SCISSORS
4. LIZARD
5. SPOCK
Enter your choice: 5
Your Move: SPOCK
Computer Move: SPOCK
Round Tie.

Final Score
You: 0
Computer: 4
Computer wins 4-0
PS E:\java\25aiml044>
```

Practical 2

Part A — Practice Programs

1. Smart thermostat

CODE:

```
import java.util.*;


public class Thermostat
{
    @SuppressWarnings("unused")
```

```
private String location;
private int temperature;
private static int MIN=16;
private static int MAX=30;
private static int activeCount=0;
public Thermostat(String l,int t)
{
   if(t>MIN && t<MAX)
   {
      temperature=t;
   }
   else
   {
      temperature=22;
   }
   activeCount++;

   this.location=l;

}
public Thermostat(String l)
{
   this(l,22);

}
```

```
public  void raise()
{
   if(temperature<MAX)
   {
      temperature+=1;
   }
   else
   {
      System.out.println("Already at maximum(30)");
   }

}
public void lower()
{
   if(temperature>MIN)
   {
      temperature--;
   }
   else
   {
      System.out.println("Already at minimum(16)");

   }
}
public int getTemperature()
```

```
    {
        return temperature;
    }
    public static int getActiveCount()
    {
        return activeCount;
    }
    public static void main(String[] args)
    {

        @SuppressWarnings("resource")
        Scanner s1 = new Scanner(System.in);
        int temp = s1.nextInt();

        Thermostat t1 =  new Thermostat("room");
        Thermostat t2 =  new Thermostat("hall", temp);

        for(int i=0;i<10;i++)
        {
            t1.raise();
            t2.raise();
            System.out.println("Temp from t1 =
"+t1.getTemperature()+" Temp from t2 = "+t2.getTemperature());
        }
        for(int j=0;j<20;j++)
```

```
        {
            t1.lower();
            t2.lower();
            System.out.println("Temp from t1 =
"+t1.getTemperature()+" Temp from t2 = "+t2.getTemperature());

        }
        System.out.println(Thermostat.getActiveCount());

    }

}
```

OUTPUT:

```
PS D:\java\git java\JAVApracticals\PB02\Q1> java Thermostat.java
12
Temp from t1 = 23 Temp from t2 = 23
Temp from t1 = 24 Temp from t2 = 24
Temp from t1 = 25 Temp from t2 = 25
Temp from t1 = 26 Temp from t2 = 26
Temp from t1 = 27 Temp from t2 = 27
Temp from t1 = 28 Temp from t2 = 28
Temp from t1 = 29 Temp from t2 = 29
Temp from t1 = 30 Temp from t2 = 30
Already at maximum(30)
Already at maximum(30)
Temp from t1 = 30 Temp from t2 = 30
Already at maximum(30)
Already at maximum(30)
Temp from t1 = 30 Temp from t2 = 30
Temp from t1 = 29 Temp from t2 = 29
Temp from t1 = 28 Temp from t2 = 28
Temp from t1 = 27 Temp from t2 = 27
Temp from t1 = 26 Temp from t2 = 26
Temp from t1 = 25 Temp from t2 = 25
Temp from t1 = 24 Temp from t2 = 24
Temp from t1 = 23 Temp from t2 = 23
Temp from t1 = 22 Temp from t2 = 22
Temp from t1 = 21 Temp from t2 = 21
Temp from t1 = 20 Temp from t2 = 20
Temp from t1 = 19 Temp from t2 = 19
Temp from t1 = 18 Temp from t2 = 18
Temp from t1 = 17 Temp from t2 = 17
Temp from t1 = 16 Temp from t2 = 16
Temp from t1 = 17 Temp from t2 = 17
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
Already at minimum(16)
Already at minimum(16)
Temp from t1 = 16 Temp from t2 = 16
2
PS D:\java\git java\JAVApracticals\PB02\Q1>
```

2. Cinema show

CODE:

```
import java.util.*;

public class CinemaShow
{
    @SuppressWarnings("unused")
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked=0;
    public CinemaShow(String t,int c)
    {
        title=t;
        capacity=c;
        seatsAvailable=capacity;
    }
    public CinemaShow(String t)
    {
        this(t,100);
    }

    public boolean book (int n)
    {
        if(n<=seatsAvailable)
        {
```

```
            seatsAvailable-=n;
            totalBooked+=n;
            return true;
        }
        else
        {
            return false;
        }
    }
    public void cancel(int n)
    {
        seatsAvailable+=n;
        if(seatsAvailable>capacity)
        {
            seatsAvailable=capacity;
        }
    }
    public int getSeatsAvailable()
    {
        return seatsAvailable;
    }
    public static int getTotalBooked()
    {
        return totalBooked;
    }
```

```
public static void main(String[] args)
{

    Scanner s= new Scanner(System.in);



    int cap;
    System.out.print("Enter capcaity of cinema:");
    cap=s.nextInt();
    s.nextLine();

    String t;
    System.out.print("Enter movie:");
    t=s.nextLine();

     CinemaShow c= new CinemaShow(t,cap);

    while (true){
        System.out.println("1. Book ticket");
        System.out.println("2. Cancel ticket");
        System.out.println("3. Show Available Seats");
        System.out.println("4. Exit");
        System.out.print("Enter your choice:");
        int choice=s.nextInt();
        switch ((choice))
```

```
        {
            case 1:
                System.out.print("Enter number of tickets:");
                int ti=s.nextInt();
                boolean result = c.book(ti);
                System.out.println("Booking Successful:" + result);
                System.out.println("Seats Available:" +
c.getSeatsAvailable());
                break;

            case 2:
                 System.out.print("Enter number of tickets to cancel:");
                int cancel = s.nextInt();

                c.cancel(cancel);

                System.out.println("Seats Available:" +
c.getSeatsAvailable());
                break;
            case 3:
                System.out.println("Seats Available:" +
c.getSeatsAvailable());
                break;
            case 4:
```

```
                System.out.println("Total Successful Bookings: " +
CinemaShow.getTotalBooked());


                s.close();
                return;
            default:
                System.out.println("Invalid Choice!");
        }


    }


  }
}
```

OUTPUT:

```
PS D:\java\git java\JAVApracticals\PB02\Q2> java CinemaShow.java
Enter capcaity of cinema:50
Enter movie:xyz
1. Book ticket
2. Cancel ticket
3. Show Available Seats
4. Exit
Enter your choice:1
Enter number of tickets:3
Booking Successful:true
Seats Available:47
1. Book ticket
2. Cancel ticket
3. Show Available Seats
4. Exit
Enter your choice:3
Seats Available:47
1. Book ticket
2. Cancel ticket
3. Show Available Seats
4. Exit
Enter your choice:2
Enter number of tickets to cancel:2
Seats Available:49
1. Book ticket
2. Cancel ticket
3. Show Available Seats
4. Exit
Enter your choice:4
Total Successful Bookings: 3
PS D:\java\git java\JAVApracticals\PB02\Q2>
```

3. Parking lot

CODE:

```
import java.util.*;


public class ParkingLot
{
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue=0;
```

```
public ParkingLot(int twoCap, int fourCap)
{
    this.twoCap = twoCap;
    this.fourCap = fourCap;
    twoWheelers = 0;
    fourWheelers = 0;
}
public void park(String type)
{
    switch (type.toUpperCase())
    {
        case "TWO":

            if(twoWheelers<twoCap)
            {
                twoWheelers++;
                revenue+=20;
            }
            else
            {
                System.out.println("FULL");
            }

            break;
        case "FOUR":
```

```
            if(fourWheelers<fourCap)
            {
                fourWheelers++;
                revenue+=40;
            }
            else
            {
                System.out.println("FULL");
            }
            break;

        default:
            System.out.println("Invalid type");

    }
}
public void leave(String t)
{
    switch (t.toUpperCase())
    {
        case "TWO":

            if(twoWheelers>0)
            {
```

```
            twoWheelers--;

        }
        else
        {
            System.out.println("Empty");
        }

        break;
    case "FOUR":

        if(fourWheelers>0)
        {
            fourWheelers--;

        }
        else
        {
            System.out.println("Empty");
        }
        break;

    default:
        System.out.println("Invalid type");
```

```
    }

}

 public int getTwoWheelers()
{
    return twoWheelers;
}

public int getFourWheelers()
{
    return fourWheelers;
}

public static long getRevenue()
{
    return revenue;
}
public static void main(String[] args)
{
    Scanner s=new Scanner(System.in);
     System.out.print("Enter two-wheeler capacity:");
    int two = s.nextInt();

    System.out.print("Enter four-wheeler capacity:");
```

```
        int four = s.nextInt();

        ParkingLot p= new ParkingLot(two, four);
        while(true)
        {
            System.out.println("1. Park Vehicle");
            System.out.println("2. Leave Vehicle");
            System.out.println("3. Show Status");
            System.out.println("4. Exit");

            System.out.print("Enter choice: ");
            int choice = s.nextInt();

            switch (choice)
            {

                case 1:
                    System.out.print("Enter vehicle type: ");
                    String type = s.next();
                    p.park(type);

                    System.out.println("Two-Wheelers:" +
p.getTwoWheelers());
                    System.out.println("Four-Wheelers:" +
p.getFourWheelers());
```

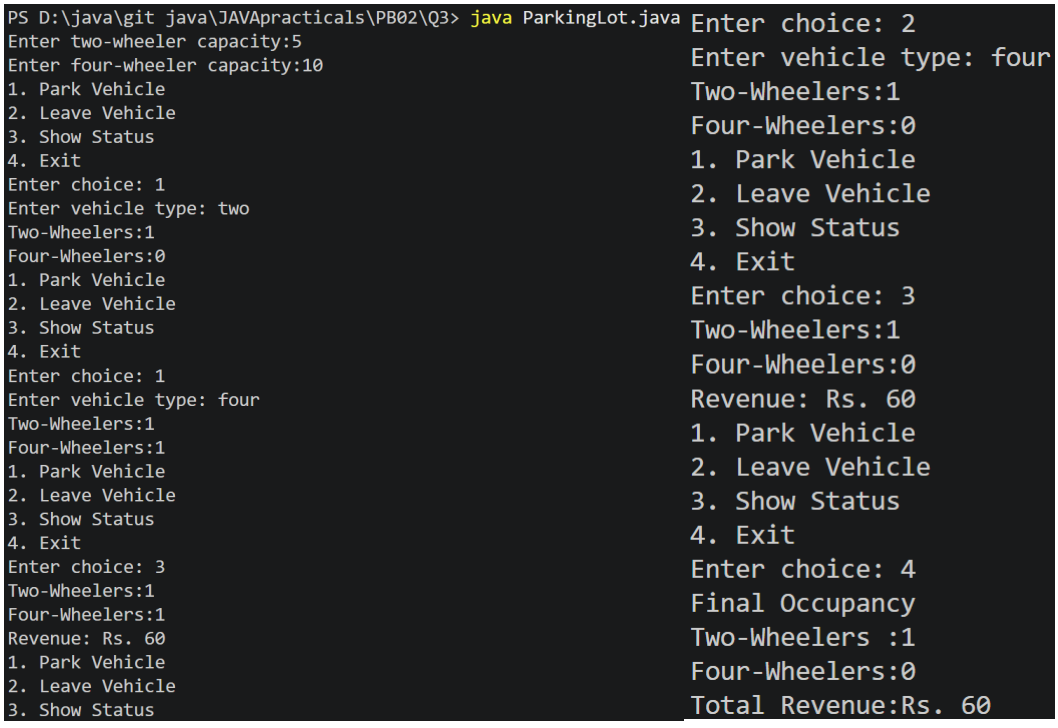
```
                break;
            case 2:
                System.out.print("Enter vehicle type: ");
                String ty = s.next();
                p.leave(ty);
                System.out.println("Two-Wheelers:" +
p.getTwoWheelers());
                System.out.println("Four-Wheelers:" +
p.getFourWheelers());
                break;
            case 3:
                 System.out.println("Two-Wheelers:" +
p.getTwoWheelers());
                System.out.println("Four-Wheelers:" +
p.getFourWheelers());
                System.out.println("Revenue: Rs. " +
ParkingLot.getRevenue());
                break;


            case 4:
                System.out.println("Final Occupancy");
                System.out.println("Two-Wheelers :" +
p.getTwoWheelers());
                System.out.println("Four-Wheelers:" +
p.getFourWheelers());
                System.out.println("Total Revenue:Rs. " +
ParkingLot.getRevenue());
```

```
                s.close();
            default:
                System.out.println("Invalid Choice");


        }
    }
}
}
```

OUTPUT:



```
PS D:\java\git java\JAVApracticals\PB02\Q3> java ParkingLot.java
Enter two-wheeler capacity:5
Enter four-wheeler capacity:10
1. Park Vehicle
2. Leave Vehicle
3. Show Status
4. Exit
Enter choice: 1
Enter vehicle type: two
Two-Wheelers:1
Four-Wheelers:0
1. Park Vehicle
2. Leave Vehicle
3. Show Status
4. Exit
Enter choice: 1
Enter vehicle type: four
Two-Wheelers:1
Four-Wheelers:1
1. Park Vehicle
2. Leave Vehicle
3. Show Status
4. Exit
Enter choice: 3
Two-Wheelers:1
Four-Wheelers:1
Revenue: Rs. 60
1. Park Vehicle
2. Leave Vehicle
3. Show Status
Enter choice: 2
Enter vehicle type: four
Two-Wheelers:1
Four-Wheelers:0
1. Park Vehicle
2. Leave Vehicle
3. Show Status
4. Exit
Enter choice: 3
Two-Wheelers:1
Four-Wheelers:0
Revenue: Rs. 60
1. Park Vehicle
2. Leave Vehicle
3. Show Status
4. Exit
Enter choice: 4
Final Occupancy
Two-Wheelers :1
Four-Wheelers:0
Total Revenue:Rs. 60
```

Practical 3

Part A	
1)	**AIM:** Distinct points (Point.java + Driver.java with main) (a) Create class Point with private int x, y and a constructor. (b) Override toString() to return “(x, y)”. (c) Override equals(Object) to be true when both x and y match, and hashCode() to agree (e.g. Objects.hash(x, y)). (d) In Driver.main: build a Point[] with some repeated coordinates. (e) Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not. (f) Print “Distinct: N”. Expected output: 5 points with 2 repeats → “Distinct: 3”
	Point.java

Code	import java.util.*;
	
	public class Point {
	private int x;
	private int y;
	
	public Point(int x, int y) {
	this.x = x;
	this.y = y;
	}
	
	@Override
	public String toString() {
	return "(" + x + "," + y + ")";
	}
	
	@Override
	public boolean equals(Object o){
	if(this==o){
	return true;
	}
	if(!(o instanceof Point)){
	return false;
	}
	Point p= (Point)o;
	return this.x==p.x && this.y==p.y;
	}
	
	@Override
	public int hashCode() {
	return Objects.hash(x, y);
	}
	
	}

	Driver.java

Code	public class Driver {
	public static void main(String []args){
	Point[] point={new Point(1,2),new Point(3,4),new Point(5,6),new Point(3,4)};
	
	int distinctcount = 0;
	for (int i = 0; i < point.length; i++) {
	boolean seenBefore = false;
	for (int j = 0; j < i; j++) {
	if (point[i].equals(point[j])) {
	seenBefore = true;
	break;
	}
	}
	if (!seenBefore) distinctcount++;
	}
	
	System.out.println("Distinct:" + distinctcount);
	}
	}

Output	Distinct:3
2)	**AIM:** Duplicate card check (Card.java + Driver.java) (a) Create class Card with private String rank, suit and a constructor. (b) Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts"). (c) Override equals()/hashCode() by rank + suit. (d) In Driver.main: add cards to a Card[] one at a time; before adding each, use equals() to check it against the earlier cards. (e) Print "Duplicate found: " the first time a repeat appears. Expected output: a second "Ace of Spades" → "Duplicate found: Ace of Spadescalls, printing the result and seats after each, then print getTotalBooked(). Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings
	Card.java

Code	import java.util.Objects;
	
	public class Fraction {
	private int num, den;
	
	public Fraction(int num, int den) {

```
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    @Override
    public String toString() {
        return num + "/" + den;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o){
            return true;
        }
        if (!(o instanceof Fraction)){
            return false;
        }
        Fraction f = (Fraction) o;
        return num == f.num && den == f.den;
    }

    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Driver.java

```
import java.util.ArrayList;
import java.util.List;

public class Driver {
    public static void main(String[] args) {
        List<Card> cards = new ArrayList<>();
        Card[] toAdd = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("Ace", "Spades")   // duplicate
        };

        for (Card c : toAdd) {
            boolean isDuplicate = false;
            for (Card existing : cards) {
                if (existing.equals(c)) {
                    isDuplicate = true;
```

```
                    break;
                }
            }
            if (isDuplicate) {
                System.out.println("Duplicate found: " + c);
                break; // stop at first repeat, as spec says "the first time"
            }
            cards.add(c);
        }
    }
}
```

Output

```
Duplicate found: Ace of Spades
```

Q3)

AIM: Fractions (Fraction.java + Driver.java)
(a) Create class Fraction with private int num, den.
(b) In the constructor, reduce to lowest terms: compute g = gcd(num, den) and divide both by g.
(c) Override toString() to return num + "/" + den.
(d) Override equals()/hashCode() using the reduced num and den.
(e) In Driver.main: create several equivalent fractions (1/2, 2/4, 3/6) and confirm they are all equal and print the same.
Expected output: new Fraction(2,4) prints "1/2" and equals new Fraction(1,2)

Fraction.java

Code

```
import java.util.Objects;

public class Fraction {
    private int num, den;

    public Fraction(int num, int den) {
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
    }

    private int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }

    @Override
    public String toString() {
        return num + "/" + den;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o){
```

```
            return true;
        }
        if (!(o instanceof Fraction)){
            return false;
        }
        Fraction f = (Fraction) o;
        return num == f.num && den == f.den;
    }

    @Override
    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Driver.java

Code

```
public class Driver {
    public static void main(String[] args) {
        Fraction a = new Fraction(1, 2);
        Fraction b = new Fraction(2, 4);
        Fraction c = new Fraction(3, 6);

        System.out.println(a);
        System.out.println(b);
        System.out.println(c);
        System.out.println(a.equals(b));
        System.out.println(b.equals(c));
        }
}
```

Output

```
1/2
1/2
1/2
true
true
```

Practical 4

Part A

1)	Password strength (PasswordChecker.java + Driver.java) (a) In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods. (b) Write strength(String pw) that counts how many rules pass and returns "Weak" (0–1), "Medium" (2–3) or "Strong" (4). (c) In Driver.main: test several passwords; for each, print which rules passed and the final label. Expected output: "abc" → Weak; "Abcd1234!" → Strong.
	PaccwordChecker.java

Code	public class PasswordChecker{
	public static boolean hasMinimumLength(String pw){
	return pw.length()>=8;
	}
	public static boolean hasUppercase(String pw){
	return pw.matches(".*[A-Z].*");
	}
	public static boolean hasDigit(String pw){
	return pw.matches(".*[0-9].*");
	}
	public static boolean hasSpecialCharacter(String pw){
	return pw.matches(".*[^a-zA-Z0-9].*");
	}
	public static String strength(String pw){
	int count=0;
	if(hasMinimumLength(pw)){
	count++;
	}
	if(hasUppercase(pw)){
	count++;
	}
	if(hasDigit(pw)){
	count++;
	}
	if(hasSpecialCharacter(pw)){
	count++;
	}
	if(count<=1){
	return "Weak";
	}
	else if(count<=3){
	return "Medium";
	}

```
        else{
            return "Strong";
        }
    }
}
```

Driver.java

Code

```
public class Driver {

    public static void main(String[] args) {

        String[] passwords = {
            "abc",
            "abcdefghi",
            "Abcdefgh",
            "Abcd1234",
            "Abcd1234!"
        };

        for (String pw : passwords) {

            System.out.println("Password: " + pw);

            System.out.println("Length >= 8: "
                    + PasswordChecker.hasMinimumLength(pw));

            System.out.println("Uppercase letter: "
                    + PasswordChecker.hasUppercase(pw));

            System.out.println("Digit:  "
                    + PasswordChecker.hasDigit(pw));

            System.out.println("Special character: "
                    + PasswordChecker.hasSpecialCharacter(pw));

            System.out.println("Strength:  "
                    + PasswordChecker.strength(pw));

            System.out.println(" ---------- ");
        }
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB04> cd "D:\java\git java\JAVApracticals\LAB04\Q1"
PS D:\java\git java\JAVApracticals\LAB04\Q1> javac Driver.java PasswordChecker.java
PS D:\java\git java\JAVApracticals\LAB04\Q1> java Driver.java PasswordChecker.java
Password: abc
Length >= 8: false
Uppercase letter: false
Digit: false
Special character: false
Strength: Weak

Password: abcdefghi
Length >= 8: true
Uppercase letter: false
Digit: false
Special character: false
Strength: Weak

Password: Abcdefgh
Length >= 8: true
Uppercase letter: true
Digit: false
Special character: false
Strength: Medium

Password: Abcd1234
Length >= 8: true
Uppercase letter: true
Digit: true
Special character: false
Strength: Medium

Password: Abcd1234!
Length >= 8: true
Uppercase letter: true
Digit: true
Special character: true
Strength: Strong
```

2)

AIM: Chat-log filter (ChatFilter.java + Driver.java)

(a) In Driver.main, define a String[] of log lines like "10:05 alice Hello there" (include one malformed line with fewer than 3 parts).

(b) Read a keyword from the user.

(c) For each line, split into time/user/message with split(" ", 3); skip lines that do not have 3 parts.

(d) If the message contains the keyword (case-insensitive), increment a counter and append "time user: message" to a StringBuilder.

(e) Print "Matches: N" and the built report.

Expected output: keyword "hello" over 3 lines (1 malformed) → "Matches: 1" and the matching line.

ChatFilter .java

Code

```
public class ChatFilter{
    public static String filterLogs(String[] logs, String keyword){
        int count=0;
        StringBuilder report=new StringBuilder();
        for(String line:logs){
            String[] parts = line.split(" ",3);
            if (parts.length<3) {
                continue;
            }
            String time = parts[0];
```

```
            String user = parts[1];
            String message = parts[2];
            if(message.toLowerCase().contains(keyword.toLowerCase())){
                count++;
                report.append(time)
                    .append(" ")
                    .append(user)
                    .append(": ")
                    .append(message)
                    .append("\n");
            }
        }
        return "Matches: " + count + "\n" + report;
    }
}
```

Driver1.java

```
import java.util.Scanner;

public class Driver1{
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:06 bob How are you?",
            "10:07 InvalidLine"
        };
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();
        String result = ChatFilter.filterLogs(logs, keyword);
        System.out.println(result);
        sc.close();
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB04> cd "D:\java\git java\JAVApracticals\LAB04\Q2"
PS D:\java\git java\JAVApracticals\LAB04\Q2> javac Driver.java ChatFilter.java
PS D:\java\git java\JAVApracticals\LAB04\Q2> java Driver.java ChatFilter.java
Enter keyword: black
Matches: 0

PS D:\java\git java\JAVApracticals\LAB04\Q2> java Driver.java ChatFilter.java
Enter keyword: Hello
Matches: 1
10:05 alice: Hello there

PS D:\java\git java\JAVApracticals\LAB04\Q2>
```

Q3)	**AIM:** Template filler (TemplateFiller.java + Driver.java) (a) In Driver.main, define a template like "Dear {name}, order {id} ships {date}." and two parallel arrays: names {"name", "id"} and values {"Riya", "A07"} (note {date} is not supplied). (b) Compile a Pattern for \{(\w+)\} and get a Matcher on the template. (c) Walk the matches with a StringBuilder: for each {placeholder}, look up its value in the names array and replace with the value, or with "[?]" if not found.

	(d) Print the filled template. Expected output: the unsupplied {date} becomes [?].
	TemplateFiller .java

Code	import java.util.regex.Matcher;
	import java.util.regex.Pattern;
	public class TemplateFiller{
	  public static String fillTemplate(String template, String[] names, String[] values){
	  Pattern pattern=Pattern.compile("\\{(\\w+)\\}");
	  Matcher matcher=pattern.matcher(template);
	  StringBuilder result = new StringBuilder();
	  while (matcher.find()){
	  String placeholder = matcher.group(1);
	  String replacement = "[?]";
	  for(int i=0;i<names.length;i++) {
	  if(names[i].equals(placeholder)) {
	  replacement=values[i];
	  break;
	  }
	  }
	  matcher.appendReplacement(result, Matcher.quoteReplacement(replacement));
	  }
	  matcher.appendTail(result);
	  return result.toString();
	  }
	}

	Driver2.java

Code	public class Driver2{
	  public static void main(String[] args){
	  String template = "Dear {name}, order {id} ships {date}.";
	  String[] names = {"name", "id"};
	  String[] values = {"Riya", "A07"};
	  String result = TemplateFiller.fillTemplate(
	  template, names, values);
	  System.out.println(result);
	  }
	}

Output	PS D:\java\git java\JAVApracticals\LAB04> cd "D:\java\git java\JAVApracticals\LAB04\Q3" PS D:\java\git java\JAVApracticals\LAB04\Q3> java Driver.java TemplateFiller.java Dear MANAV, order A07 ships [?]. PS D:\java\git java\JAVApracticals\LAB04\Q3>

Practical 5

Part A	
1)	**AIM1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total,**

Code	import java.util.*;
	abstract class Shape{
	public abstract float area();
	}
	class Circle extends Shape{
	public float radius;
	public Circle(int r){
	radius=r;
	}
	public float area(){
	return 3.14f*radius*radius;
	}
	}
	class Rectangle extends Shape{
	public float lenght;
	public float breadth;
	
	public Rectangle(float l, float b){
	lenght = l;
	breadth = b;
	}
	public float area(){
	return lenght * breadth;
	}
	
	}
	class Triangle extends Shape{
	public float base;
	public float height;
	
	public Triangle(float b, float h){
	base= b;
	height = h;
	}
	public float area(){

```
            return 0.5f*base*height;
        }
}
public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {new Circle(5),new Rectangle(4, 6),new Triangle(3, 8),new
Circle(3)};

        double total_area = 0;
        double largest_area = 0;

        for (Shape shape : shapes) {

            double current_area = shape.area();

            System.out.printf("Area: %.2f%n", current_area);
            total_area += current_area;

            if (current_area > largest_area) {
                largest_area = current_area;
            }
        }

        System.out.printf("%nTotal Area: %.2f%n", total_area);
        System.out.printf("Largest Area: %.2f%n", largest_area);

    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB05> cd "D:\java\git java\JAVApracticals\LAB05\Q1"
PS D:\java\git java\JAVApracticals\LAB05\Q1> javac main.java
PS D:\java\git java\JAVApracticals\LAB05\Q1> java main.java
Area: 78.50
Area: 24.00
Area: 12.00
Area: 28.26

Total Area: 142.76
Largest Area: 78.50
PS D:\java\git java\JAVApracticals\LAB05\Q1>
```

2) **AIM: Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.**

Code

```
import java.util.*;

abstract class Employee {
```

```
    public String employee_name;
    public int employee_id;

    public Employee(String name, int id) {
        employee_name = name;
        employee_id = id;
    }

    public abstract float monthlySalary();
}

class FullTime extends Employee {

    public float salary;

    public FullTime(String name, int id) {
        super(name, id);
        salary = 100000;
    }

    public float monthlySalary() {
        return salary;
    }
}

class PartTime extends Employee {

    public int hour;
    public float rate;

    public PartTime(String name, int id, int h, float r) {
        super(name, id);
        hour = h;
        rate = r;
    }

    public float monthlySalary() {
        return rate * hour;
    }
}

class Intern extends Employee {

    public float salary;

    public Intern(String name, int id) {
        super(name, id);
        salary = 10000;
    }

    public float monthlySalary() {
```

```
        return salary;
    }
}

public class Main {

    public static void main(String[] args) {

        Employee[] employees = {
            new FullTime("Ansh", 101),
            new PartTime("Raj", 102, 80, 300),
            new Intern("Jay", 103),
            new FullTime("Karan", 104),
            new Intern("Dev", 105)
        };

        float total_salary = 0;

        for (Employee employee : employees) {
            float salary = employee.monthlySalary();

            System.out.println("Employee ID: " + employee.employee_id +", Name:" +
employee.employee_name +", Salary: " + salary);

            if (employee instanceof Intern) {
                System.out.println("Employee is an Intern.");
            }

            total_salary = total_salary +salary;
        }

        System.out.println("Total Monthly Payroll:" + total_salary);
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB05\Q1> cd "D:\java\git java\JAVApracticals\LAB05\Q2"
PS D:\java\git java\JAVApracticals\LAB05\Q2> javac Main.java
PS D:\java\git java\JAVApracticals\LAB05\Q2> java Main.java
Employee ID: 101, Name:Ansh, Salary: 100000.0
Employee ID: 102, Name:Raj, Salary: 24000.0
Employee ID: 103, Name:Jay, Salary: 10000.0
Employee is an Intern.
Employee ID: 104, Name:Karan, Salary: 100000.0
Employee ID: 105, Name:Dev, Salary: 10000.0
Employee is an Intern.
Total Monthly Payroll:244000.0
PS D:\java\git java\JAVApracticals\LAB05\Q2>
```

Q3) **AIM:** Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself

Code

```
abstract class Media {
```

```
    public String media_name;
    public int media_id;
    public int late_days;

    public Media(String n, int i, int d) {
        media_name = n;
        media_id = i;
        late_days = d;
    }
    public abstract float late_fee();
}

class Book extends Media {

    public Book(String name, int id, int days) {
        super(name, id, days);
    }

    public float late_fee() {
        return late_days * 2;
    }
}

class DVD extends Media {

    public DVD(String name, int id, int days) {
        super(name, id, days);
    }

    public float late_fee() {
        return late_days * 5;
    }
}

class Game extends Media {

    public Game(String name, int id, int days) {
        super(name, id, days);
    }

    public float late_fee() {
        return late_days*8;
    }
}

public class Main {

    public static void main(String[] args) {

        Media[] returned_media = {new Book("Java Programming", 101, 3),new
DVD("Avengers", 102, 2),new Game("Minecraft", 103, 5),
```

```
            new Book("Data Structures", 104, 4),new DVD("Interstellar", 105, 1)};

        float total_fee=0;

        for (Media media:returned_media) {

            float fee = media.late_fee();

            System.out.println("Media ID:" + media.media_id +",Name:" +
media.media_name +",Late Days:" + media.late_days +",Late Fee:Rs." + fee);

            total_fee = total_fee + fee;
        }
        System.out.println("Total Late Fees:Rs." + total_fee);
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB05\Q3> javac main.java
PS D:\java\git java\JAVApracticals\LAB05\Q3> java main.java
Media ID:101,Name:Java Programming,Late Days:3,Late Fee:Rs.6.0
Media ID:102,Name:Avengers,Late Days:2,Late Fee:Rs.10.0
Media ID:103,Name:Minecraft,Late Days:5,Late Fee:Rs.40.0
Media ID:104,Name:Data Structures,Late Days:4,Late Fee:Rs.8.0
Media ID:105,Name:Interstellar,Late Days:1,Late Fee:Rs.5.0
Total Late Fees:Rs.69.0
PS D:\java\git java\JAVApracticals\LAB05\Q3>
```

Practical 6

Part A

1) **Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.**

Code

```
interface Switchable
{

    void on();

    void off();

    default void toggle()
{
        on();
```

```
    }
}

class Fan implements Switchable
 {

    public void on() {
        System.out.println("Fan is ON");
    }

    public void off() {
        System.out.println("Fan is OFF");
    }
}

class Light implements Switchable
{

    public void on() {
        System.out.println("Light is ON");
    }

    public void off() {
        System.out.println("Light is OFF");
    }
}

@FunctionalInterface
interface SwitchPermission
{
    boolean maySwitchOn(Switchable device, int hour);
}

public class RemoteControl
{

    public static void main(String[] args)
{

        Switchable[] devices =
       {
            new Fan(),
            new Light()
        };

        System.out.println("Toggling Devices:");

        for (Switchable device : devices)
       {
            device.toggle();
        }

        SwitchPermission permission1 = new SwitchPermission() {

            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
```

```
        }
    };

    SwitchPermission permission2 =
            (device, hour) -> hour >= 6 && hour <= 22;

    int hour = 10;

    System.out.println("\nAnonymous Class:");
    System.out.println(
        permission1.maySwitchOn(devices[0], hour)
    );

    System.out.println("\nLambda:");
    System.out.println(
        permission2.maySwitchOn(devices[1], hour)
    );
  }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB06\Q1> javac RemoteControl.java
PS D:\java\git java\JAVApracticals\LAB06\Q1> java RemoteControl.java
Toggling Devices:
Fan is ON
Light is ON

Anonymous Class:
true

Lambda:
true
```

2) **Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.**

Code

```
@FunctionalInterface
interface Notifier {
    void send(String message);
}

interface Urgent {
}

class UrgentNotifier implements Notifier, Urgent {

    private Notifier notifier;

    UrgentNotifier(Notifier notifier) {
        this.notifier = notifier;
    }

    public void send(String message) {
        notifier.send(message);
    }
```

```
}

public class NotificationSender {

    public static void main(String[] args) {

        Notifier email = message ->
                System.out.println("Email: " + message);

        Notifier sms = message ->
                System.out.println("SMS: " + message);

        Notifier urgentEmail = new UrgentNotifier(email);

        Notifier[] senders = {
            urgentEmail,
            sms
        };

        String message = "Exam tomorrow at 10 AM";

        for (Notifier sender : senders) {

            sender.send(message);

            if (sender instanceof Urgent) {
                sender.send(message);
            }
        }
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB06\Q1> cd "D:\java\git java\JAVApracticals\LAB06\Q2"
PS D:\java\git java\JAVApracticals\LAB06\Q2> javac NotificationSender.java
PS D:\java\git java\JAVApracticals\LAB06\Q2> java NotificationSender.java
Email: Exam tomorrow at 10 AM
Email: Exam tomorrow at 10 AM
SMS: Exam tomorrow at 10 AM
```

Q3) (For Advanced Learners – home) Build a small discount engine where each rule is a lambda the user picks at runtime and applies to a list of prices; organise it into packages and ship it as a runnable JAR.

Code

DiscountRule

```
package discount;

@FunctionalInterface
public interface DiscountRule {
    double apply(double price);
}
```

Main.java

```
import discount.DiscountRule;
import java.util.*;

public class Main {
```

```
public static void main(String[] args) {

    List<Double> prices = Arrays.asList(
            1000.0,
            2000.0,
            3000.0,
            5000.0
    );

    Scanner sc = new Scanner(System.in);

    System.out.println("Discount Engine");
    System.out.println("1. 10% Discount");
    System.out.println("2. 20% Discount");
    System.out.println("3. Flat Rs.500 Discount");

    System.out.print("Choose discount: ");
    int choice = sc.nextInt();

    DiscountRule rule;

    switch (choice) {

        case 1:
            rule = price -> price * 0.90;
            break;

        case 2:
            rule = price -> price * 0.80;
            break;

        case 3:
            rule = price -> price - 500;
            break;

        default:
            System.out.println("Invalid choice");
            sc.close();
            return;
    }

    System.out.println("\nOriginal Price -> Final Price");

    for (double price : prices) {

        double finalPrice = rule.apply(price);

        if (finalPrice < 0) {
            finalPrice = 0;
        }

        System.out.println(
                "Rs." + price + " -> Rs." + finalPrice
        );
    }
```

```
        sc.close();
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB06\Q3> javac discount\DiscountRule.java Main.java
PS D:\java\git java\JAVApracticals\LAB06\Q3> java Main
Discount Engine
1. 10% Discount
2. 20% Discount
3. Flat Rs.500 Discount
Choose discount: 3

Original Price -> Final Price
Rs.1000.0 -> Rs.500.0
Rs.2000.0 -> Rs.1500.0
Rs.3000.0 -> Rs.2500.0
Rs.5000.0 -> Rs.4500.0
```

PRACTICAL - 07

Part A — Practice Programs (Hour 1)

PROBLEM1	**1. Form validator: define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.**
CODE	

```
import java.lang.annotation.*;
import java.lang.reflect.Field;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
   int value();
}

class SignupForm {

   @NotBlank
   @MaxLength(10)
   String username;

   @NotBlank
   @MaxLength(20)
   String email;

   SignupForm(String username, String email) {
      this.username = username;
      this.email = email;
   }
}

public class Main {

   public static void main(String[] args) throws Exception {

      SignupForm form = new SignupForm("", "abcdefghijklmnop@gmail.com");

      List<String> errors = new ArrayList<>();

      for (Field field : form.getClass().getDeclaredFields()) {

         field.setAccessible(true);
```

```
            String value = (String) field.get(form);

            // Check NotBlank
            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null || value.trim().isEmpty()) {
                    errors.add(field.getName() + " cannot be blank");
                }
            }

            // Check MaxLength
            if (field.isAnnotationPresent(MaxLength.class)) {

                MaxLength max = field.getAnnotation(MaxLength.class);

                if (value != null && value.length() > max.value()) {
                    errors.add(field.getName()
                            + " is too long. Maximum "
                            + max.value() + " characters allowed");
                }
            }
        }

        if (errors.isEmpty()) {
            System.out.println("Form is valid");
        } else {
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB07> cd "D:\java\git java\JAVApracticals\LAB07\Q1"
PS D:\java\git java\JAVApracticals\LAB07\Q1> javac main.java
PS D:\java\git java\JAVApracticals\LAB07\Q1> java main.java
username cannot be blank
email is too long. Maximum 20 characters allowed
PS D:\java\git java\JAVApracticals\LAB07\Q1>
```

PROBLEM2

2. Mini test runner: define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

CODE

```
import java.lang.annotation.*;
import java.lang.reflect.Method;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class MyTests {

   @Run
   public void test1() {
      System.out.println("Test 1 executed");
   }

   @Run
   public void test2() {
      System.out.println("Test 2 executed");
   }

   public void normalMethod() {
      System.out.println("Normal method");
   }
}

public class Mini {

   public static void main(String[] args) throws Exception {

      MyTests obj = new MyTests();

      int count = 0;

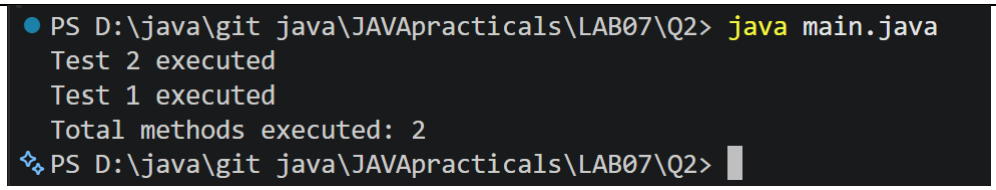
      Method[] methods = obj.getClass().getDeclaredMethods();

      for (Method method : methods) {

         if (method.isAnnotationPresent(Run.class)
               && method.getParameterCount() == 0) {

            method.invoke(obj);
            count++;
         }
      }

      System.out.println("Total methods executed: " + count);
   }
}
```

OUTPUT	
PROBLEM3	**3. (For Advanced Learners – home) Define a @Column(name) annotation and, by reflection, populate an object's fields from a header row and a data row by matching column names; decide how to handle a missing column.**

CODE	import java.lang.annotation.*;
	import java.lang.reflect.Field;
	
	@Retention(RetentionPolicy.RUNTIME)
	@Target(ElementType.FIELD)
	@interface Column {
	String name();
	}
	
	class Student {
	
	@Column(name = "id")
	String id;
	
	@Column(name = "name")
	String name;
	
	@Column(name = "email")
	String email;
	}
	
	public class Columns {
	
	public static void main(String[] args) throws Exception {
	
	String[] headers = {"id", "name", "email"};
	String[] data = {"101", "Om", "om@gmail.com"};
	
	Student student = new Student();
	
	Field[] fields = student.getClass().getDeclaredFields();
	
	for (Field field : fields) {
	
	if (field.isAnnotationPresent(Column.class)) {
	
	Column column = field.getAnnotation(Column.class);

```
            for (int i = 0; i < headers.length; i++) {

                if (headers[i].equals(column.name())) {

                    field.setAccessible(true);
                    field.set(student, data[i]);

                    break;
                }
            }
        }
    }

    System.out.println("ID: " + student.id);
    System.out.println("Name: " + student.name);
    System.out.println("Email: " + student.email);
  }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB07\Q3> cd "D:\java\git java\JAVApracticals\LAB07\Q3"
PS D:\java\git java\JAVApracticals\LAB07\Q3> javac main.java
PS D:\java\git java\JAVApracticals\LAB07\Q3> java main.java
ID: 101
Name: Bhavya
Email: bhavya@gmail.com
PS D:\java\git java\JAVApracticals\LAB07\Q3>
```

PRACTICAL - 08

Part A — Practice Programs (Hour 1)

PROBLEM1

1. Guarded calculator: read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

CODE

```
import java.util.Scanner;

public class Calculator {

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      try {
         System.out.print("Enter first number: ");
         double num1 = sc.nextDouble();

         System.out.print("Enter second number: ");
```

```
        double num2 = sc.nextDouble();

        if (num1 < 0 || num2 < 0) {
            throw new InvalidNumberException("Negative numbers are not allowed!");
        }

        if(num1==0 || num2==0){
            throw new InvalidNumberException("Multiplication By Zero is not allowed!");
        }

        System.out.print("Enter operator (+, -, *, /): ");
        char op = sc.next().charAt(0);

        System.out.println("You entered: " + num1 + " " + op + " " + num2);

        double result = 0;

        if (op == '+') {
            result = num1 + num2;
        } else if (op == '-') {
            result = num1 - num2;
        } else if (op == '*') {
            result = num1 * num2;
        } else if (op == '/') {
            if (num2 == 0) {
                throw new DivideByZeroException("Cannot divide by zero!");
            }
            result = num1 / num2;
        } else {
            throw new InvalidNumberException("Invalid operator entered!");
        }

        System.out.println(num1 + " " + op + " " + num2 + " = " + result);

    } catch (DivideByZeroException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (InvalidNumberException e) {
        System.out.println("Error: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Unexpected error: " + e.getMessage());
    } finally {
        System.out.println("Exiting the program.");
        sc.close();
    }
  }
}

class DivideByZeroException extends Exception {

  public DivideByZeroException(String message) {
    super(message);
```

```
    }
}

class InvalidNumberException extends Exception {

    public InvalidNumberException(String message) {
        super(message);
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB08\Q1> java Calculator.java
Enter first number: 10
Enter second number: 20
Enter operator (+, -, *, /): +
You entered: 10.0 + 20.0
10.0 + 20.0 = 30.0
Exiting the program.
PS D:\java\git java\JAVApracticals\LAB08\Q1>
```

PROBLEM2

2. Stock issue: a Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.

CODE

```
import java.util.Scanner;

class outofstockexception extends Exception {

    private int shortage;

    public outofstockexception(int shortage) {
        super("out of stock! shortage: " + shortage);
        this.shortage = shortage;
    }

    public int getshortage() {
        return shortage;
    }
}

class invalidquantityexception extends Exception {

    public invalidquantityexception() {
        super("invalid quantity! must be greater than 0.");
    }
}

class store {

    private int stock;
```

```
    public store(int stock) {
        this.stock = stock;
    }

    public void issue(String item, int qty)
            throws outofstockexception, invalidquantityexception {

        if (qty <= 0) {
            throw new invalidquantityexception();
        }

        if (qty > stock) {
            int shortage = qty - stock;
            throw new outofstockexception(shortage);
        }

        stock -= qty;

        System.out.println("issued: " + qty + " " + item);
        System.out.println("left in stock: " + stock);
    }
}

public class storemanagement {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("enter starting stock: ");
        int stock = sc.nextInt();

        store s = new store(stock);

        System.out.print("enter number of requests: ");
        int requests = sc.nextInt();
        sc.nextLine();

        for (int i = 1; i <= requests; i++) {
            System.out.println("\nrequest " + i);

            System.out.print("enter item name: ");
            String item = sc.nextLine();

            System.out.print("enter quantity: ");
            int qty = sc.nextInt();
            sc.nextLine();

            try {
                s.issue(item, qty);
            } catch (outofstockexception e) {
                System.out.println("error: " + e.getMessage());
```

```
        } catch (invalidquantityexception e) {
            System.out.println("error: " + e.getMessage());
        }
    }

    sc.close();
  }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB08> cd "D:\java\git java\JAVApracticals\LAB08\Q2"
PS D:\java\git java\JAVApracticals\LAB08\Q2> javac storemanagement.java
PS D:\java\git java\JAVApracticals\LAB08\Q2> java storemanagement.java
enter starting stock: 3
enter number of requests: 2

request 1
enter item name: book
enter quantity: 20
error: out of stock! shortage: 17

request 2
enter item name: pen
enter quantity: 5
error: out of stock! shortage: 2
PS D:\java\git java\JAVApracticals\LAB08\Q2>
```

PROBLEM3

3. (For Advanced Learners – home) Create an AutoCloseable resource and use try-with-resources to prove it always closes — even when an exception is thrown inside the block — and that the original error is the one reported. Decide what to model as the resource.

CODE

```
class MyResource implements AutoCloseable {

  void doWork() {
    System.out.println("Resource is doing some work...");
    throw new RuntimeException("Original error occurred");
  }

  @Override
  public void close() {
    System.out.println("Resource is closed");
    throw new RuntimeException("Error while closing resource");
  }
}

public class Main {

  public static void main(String[] args) {

    try (MyResource resource = new MyResource()) {

      System.out.println("Resource is opened");
      resource.doWork();
```

```
        } catch (Exception e) {

            System.out.println("Main error: " + e.getMessage());

            for (Throwable error : e.getSuppressed()) {
                System.out.println("Suppressed error: " + error.getMessage());
            }
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB08\Q3> cd "D:\java\git java\JAVApracticals\LAB08\Q3"
PS D:\java\git java\JAVApracticals\LAB08\Q3> javac main.java
PS D:\java\git java\JAVApracticals\LAB08\Q3> java main.java
Resource is opened
Resource is doing some work...
Resource is closed
Main error: Original error occurred
Suppressed error: Error while closing resource
PS D:\java\git java\JAVApracticals\LAB08\Q3>
```

CEUE203 : Object Oriented Programming 25AIML044

PRACTICAL – 9

AIM:

1. **Counter race**: several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

.**CODE:**

```
import java.util.Scanner;

class Counter {
	int count = 0;

	// Unsynchronized increment void
	increment() {
		count++;
	}

	// Synchronized increment
	synchronized void synchronizedIncrement() { count++;
	}
}

class MyThread extends Thread { Counter
	counter;
	int increments;
	boolean useSynchronization;

	MyThread(Counter counter, int increments, boolean useSynchronization) { this.counter = counter;
		this.increments = increments;
		this.useSynchronization = useSynchronization;
	}

	public void run() {
		for (int i = 0; i < increments; i++) {

			if (useSynchronization) {
				counter.synchronizedIncrement();
			} else {
				counter.increment();
			}
```

ccCSPIT - AIML	1

1L044

```
public class CounterRace {

    public static void main(String[] args) throws Exception { Scanner sc = new

        Scanner(System.in);

        System.out.print("Enter number of threads: "); int
        numberOfThreads = sc.nextInt();

        System.out.print("Enter number of increments per thread: "); int increments =
        sc.nextInt();

        System.out.print("Use synchronization? (yes/no): "); String choice = sc.next();

        boolean useSynchronization = choice.equalsIgnoreCase("yes"); Counter counter =

        new Counter();

        MyThread[] threads = new MyThread[numberOfThreads];

        // Creating threads
        for (int i = 0; i < numberOfThreads; i++) { threads[i] = new
            MyThread(
                    counter,
                    increments,
                    useSynchronization);
        }

        // Starting threads
        for (int i = 0; i < numberOfThreads; i++) { threads[i].start();
        }

        // Waiting for all threads to finish
        for (int i = 0; i < numberOfThreads; i++) { threads[i].join();
        }

        int expectedValue = numberOfThreads * increments;

        System.out.println("\n----- Result ----------------------- ");
        System.out.println("Number of Threads: " + numberOfThreads); System.out.println("Increments per
        Thread: " + increments);
        System.out.println("Expected Counter Value: " + expectedValue); System.out.println("Actual Counter
        Value: " + counter.count);
```

```
            if (counter.count == expectedValue) {
                System.out.println("Result is correct.");
            } else {
                System.out.println("Race condition occurred.");
            }

            sc.close();
        }
}
```

OUTPUT SCREENSHOT:

With synchronization :

```
PS D:\java\git java\JAVApracticals\LAB09> cd "D:\java\git java\JAVApracticals\LAB09\Q1"
PS D:\java\git java\JAVApracticals\LAB09\Q1> javac CounterRace.java
PS D:\java\git java\JAVApracticals\LAB09\Q1> java CounterRace.java
Enter number of threads: 2
Enter number of increments per thread: 2
Use synchronization? (yes/no): yes

----- Result
Number of Threads: 2
Increments per Thread: 2
Expected Counter Value: 4
Actual Counter Value: 4
Result is correct.
PS D:\java\git java\JAVApracticals\LAB09\Q1>
```

Without synchronization :

```
PS D:\java\git java\JAVApracticals\LAB09\Q1> java CounterRace.java
Enter number of threads: 3
Enter number of increments per thread: 2
Use synchronization? (yes/no): no

----- Result
Number of Threads: 3
Increments per Thread: 2
Expected Counter Value: 6
Actual Counter Value: 6
Result is correct.
PS D:\java\git java\JAVApracticals\LAB09\Q1>
```

LEARNING OUTCOME:

- How to create and use multiple threads in Java.
- How threads share common data.
- What a race condition is.

- Why race conditions can produce incorrect results.
- How the synchronized keyword prevents simultaneous access to shared data.
- How start() starts a thread and join() waits for a thread to finish.
- How synchronization helps maintain data consistency in multithreaded programs.

2**. Seat booking race**: a shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do..

CODE:

```
import java.util.Scanner; class

SeatBooking {

    int seatsLeft;
    int successfulBookings = 0;

    SeatBooking(int seats) { seatsLeft = seats;
    }

    // Booking without synchronization void
    book(String userName) {

        if (seatsLeft > 0) {

            // Small delay makes the race condition easier to observe try {
                Thread.sleep(10);
            } catch (InterruptedException e) { System.out.println(e);
            }

            seatsLeft--;
            successfulBookings++;

            System.out.println(userName + " booked a seat.");
        } else {
            System.out.println(userName + " could not book a seat.");
        }
    }

    // Booking with synchronization
    synchronized void synchronizedBook(String userName) { if (seatsLeft >

        0) {
```

```
                try {
                        Thread.sleep(10);
                } catch (InterruptedException e) { System.out.println(e);
                }

                seatsLeft--;
                successfulBookings++;

                System.out.println(userName + " booked a seat.");
            } else {
                System.out.println(userName + " could not book a seat.");
            }
        }
}

class BookingThread extends Thread { SeatBooking

        booking;
        String userName;
        boolean useSynchronization;

        BookingThread(
                SeatBooking booking, String
                userName,
                boolean useSynchronization) {
            this.booking = booking;
            this.userName = userName;
            this.useSynchronization = useSynchronization;
        }

        public void run() {

            if (useSynchronization) {
                booking.synchronizedBook(userName);
            } else {
                booking.book(userName);
            }
        }
}

public class SeatBookingRace {

        public static void main(String[] args) throws Exception { Scanner sc = new

            Scanner(System.in);
```

```
System.out.print("Enter number of seats: "); int totalSeats =
sc.nextInt();

System.out.print("Enter number of users: "); int
numberOfUsers = sc.nextInt();

System.out.print("Use synchronization? (yes/no): "); String choice = sc.next();

boolean useSynchronization = choice.equalsIgnoreCase("yes"); SeatBooking booking

= new SeatBooking(totalSeats);

BookingThread[] users = new BookingThread[numberOfUsers];

// Creating booking threads
for (int i = 0; i < numberOfUsers; i++) { String userName =

        "User-" + (i + 1);

        users[i] = new BookingThread( booking,
                        userName,
                        useSynchronization);
}

// Starting all users
for (int i = 0; i < numberOfUsers; i++) { users[i].start();
}

// Waiting for all users to finish
for (int i = 0; i < numberOfUsers; i++) { users[i].join();
}

System.out.println("\n----- Booking Result -------------------------");

System.out.println("Total Seats: "
                + totalSeats);

System.out.println("Total Users: "
                + numberOfUsers);

System.out.println("Successful Bookings: "
                +booking.successfulBookings);

System.out.println("Seats Left: "
```

```
                    + booking.seatsLeft);

            if (booking.successfulBookings > totalSeats) {
                System.out.println("Overselling occurred!");
            } else {
                System.out.println("No overselling occurred.");
            }

            sc.close();
        }
```

OUTPUT SCREENSHOT:

With synchronization :

```
PS D:\java\git java\JAVApracticals\LAB09> cd "D:\java\git java\JAVApracticals\LAB09\Q2"
PS D:\java\git java\JAVApracticals\LAB09\Q2> javac SeatBookingRace.java
PS D:\java\git java\JAVApracticals\LAB09\Q2> java SeatBookingRace.java
Enter number of seats: 5
Enter number of users: 5
Use synchronization? (yes/no): yes
User-1 booked a seat.
User-2 booked a seat.
User-3 booked a seat.
User-4 booked a seat.
User-5 booked a seat.

----- Booking Result
Total Seats: 5
Total Users: 5
Successful Bookings: 5
Seats Left: 0
No overselling occurred.
PS D:\java\git java\JAVApracticals\LAB09\Q2>
```

Without synchronization :

```
PS D:\java\git java\JAVApracticals\LAB09\Q2> java SeatBookingRace.java
Enter number of seats: 4
Enter number of users: 4
Use synchronization? (yes/no): no
User-3 booked a seat.
User-4 booked a seat.
User-2 booked a seat.
User-1 booked a seat.

----- Booking Result
Total Seats: 4
Total Users: 4
Successful Bookings: 4
Seats Left: 0
No overselling occurred.
PS D:\java\git java\JAVApracticals\LAB09\Q2>
```

LEARNING OUTCOME:

- How multiple threads can access a shared resource.
- What a race condition is in a real-world booking system.
- How race conditions can cause overselling.
- Why checking and modifying shared data must be synchronized.
- How the synchronized keyword provides thread safety.
- How multiple users can be simulated using Java threads.
- How start() is used to run threads and join() is used to wait for them.
- How synchronization ensures that the number of successful bookings never exceeds the available seats.

3**. (For Advanced Learners – home)** Split a large array of numbers across several threads that add into one shared total, show the wrong result, then fix it two ways and compare; decide how to measure the difference**.**

CODE:

```
import java.util.Scanner; class

SharedTotal {

    long total = 0;

    // Method 1: Without synchronization void
    add(long value) {
        total += value;
    }

    // Method 2: Using synchronized method
    synchronized void synchronizedAdd(long value) { total += value;
    }
}

class SumThread extends Thread { int[]

    numbers;
```

CEC5203- AIML	CSPIT-AIML	9	25AIML044

```
    int start; int end;
    SharedTotal sharedTotal; int
    method;

    SumThread(int[] numbers, int start, int end,
            SharedTotal sharedTotal, int method) {

        this.numbers = numbers; this.start =
        start;
        this.end = end;
        this.sharedTotal = sharedTotal; this.method =
        method;
    }

    public void run() {

        for (int i = start; i < end; i++) { if (method == 1) {
                // No synchronization
                sharedTotal.add(numbers[i]);

            } else {
                // Synchronized method
                sharedTotal.synchronizedAdd(numbers[i]);
            }
        }
    }
}

public class ParallelArraySum {

    public static void main(String[] args) throws Exception { Scanner sc = new

        Scanner(System.in);

        System.out.print("Enter array size: "); int size =
        sc.nextInt();

        System.out.print("Enter number of threads: "); int
        numberOfThreads = sc.nextInt();

        System.out.print("Enter maximum value of array elements: "); int maxValue =
        sc.nextInt();

        // Creating array
        int[] numbers = new int[size];
```

```
// Filling array with values
for (int i = 0; i < size; i++) {
        numbers[i] = (i % maxValue) + 1;
}

// Calculate correct answer using single thread long correctTotal =
0;

for (int number : numbers) { correctTotal +=
        number;
}

System.out.println("\nCorrect Total: " + correctTotal);

// Run both methods
for (int method = 1; method <= 2; method++) {

        SharedTotal sharedTotal = new SharedTotal();

        SumThread[] threads = new SumThread[numberOfThreads]; int part =

        size / numberOfThreads;

        long startTime = System.nanoTime();

        // Creating threads
        for (int i = 0; i < numberOfThreads; i++) { int start = i * part;

                int end;

                if (i == numberOfThreads - 1) { end = size;
                } else {
                        end = start + part;
                }

                threads[i] = new SumThread( numbers,
                                start,
                                end,
                                sharedTotal, method);
        }

        // Starting threads
```

```
                for (int i = 0; i < numberOfThreads; i++) { threads[i].start();
                }

                // Waiting for threads
                for (int i = 0; i < numberOfThreads; i++) { threads[i].join();
                }

                long endTime = System.nanoTime();

                long timeTaken = endTime - startTime;

                System.out.println("\n----- Method " + method + "---------------------------- ");

                if (method == 1) {
                        System.out.println(
                                        "Method: Without Synchronization");
                } else {
                        System.out.println(
                                        "Method: Synchronized Method");
                }

                     System.out.println(
                                    "Calculated Total: "
                                                    + sharedTotal.total);

                System.out.println(
                                "Time Taken: "
                                                + timeTaken
                                                + " nanoseconds");

                if (sharedTotal.total == correctTotal) {
                        System.out.println("Result is CORRECT.");
                } else {
                        System.out.println("Result is WRONG.");
                }
            }

            sc.close();
        }
}
```

OUTPUT SCREENSHOT:

```
PS D:\java\git java\JAVApracticals\LAB09> cd "D:\java\git java\JAVApracticals\LAB0
PS D:\java\git java\JAVApracticals\LAB09\Q3> javac ParallelArraySum.java
PS D:\java\git java\JAVApracticals\LAB09\Q3> java ParallelArraySum.java
  Enter array size: 10000
  Enter number of threads: 10
  Enter maximum value of array elements: 500

  Correct Total: 2505000

  ----- Method 1
  Method: Without Synchronization
  Calculated Total: 2437870
  Time Taken: 1068900 nanoseconds
  Result is WRONG.

  ----- Method 2
  Method: Synchronized Method
  Calculated Total: 2505000
  Time Taken: 1496300 nanoseconds
  Result is CORRECT.
PS D:\java\git java\JAVApracticals\LAB09\Q3>
```

PRACTICAL - 10

Part A — Practice Programs (Hour 1)

PROBLEM1	1. Thread-pool runner: with a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.
CODE	

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner {
   public static void main(String[] args) {
      ExecutorService executor = Executors.newFixedThreadPool(3);

      for (int i = 1; i <= 10; i++) {
         final int taskId = i;

         executor.execute(() -> {
            System.out.println(
               "Task " + taskId + " executed by "
               + Thread.currentThread().getName()
            );

            try {
               Thread.sleep(500);
            } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
            }
         });
      }

      executor.shutdown();

      try {
         if (!executor.awaitTermination(30, TimeUnit.SECONDS)) {
            executor.shutdownNow();
         }
      } catch (InterruptedException e) {
         executor.shutdownNow();
         Thread.currentThread().interrupt();
      }

      System.out.println("All tasks completed.");
   }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB10> cd "D:\java\git java\JAVApract
PS D:\java\git java\JAVApracticals\LAB10\Q1> javac ThreadPoolRunner.java
PS D:\java\git java\JAVApracticals\LAB10\Q1> java ThreadPoolRunner.java
Task 3 executed by pool-1-thread-3
Task 1 executed by pool-1-thread-1
Task 2 executed by pool-1-thread-2
Task 4 executed by pool-1-thread-3
Task 5 executed by pool-1-thread-1
Task 6 executed by pool-1-thread-2
Task 7 executed by pool-1-thread-3
Task 9 executed by pool-1-thread-1
Task 8 executed by pool-1-thread-2
Task 10 executed by pool-1-thread-2
All tasks completed.
PS D:\java\git java\JAVApracticals\LAB10\Q1>
```

PROBLEM2

2Producer–consumer: one thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

CODE

```
class SharedBuffer {
   private final int[] buffer = new int[3];
   private int count = 0;
   private int in = 0;
   private int out = 0;

   public synchronized void produce(int value)
         throws InterruptedException {

      while (count == buffer.length) {
         wait();
      }

      buffer[in] = value;
      in = (in + 1) % buffer.length;
      count++;

      System.out.println("Produced: " + value);

      notifyAll();
   }

   public synchronized int consume()
         throws InterruptedException {

      while (count == 0) {
         wait();
      }

      int value = buffer[out];
      out = (out + 1) % buffer.length;
      count--;
```

```
        System.out.println("Consumed: " + value);

        notifyAll();

        return value;
    }
}

public class ProducerConsumer {
    public static void main(String[] args)
            throws InterruptedException {

        SharedBuffer buffer = new SharedBuffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++) {
                    buffer.produce(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 10; i++) {
                    buffer.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("Production and consumption completed.");
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB10\Q1> cd "D:\java\git java\JAVApracticals\LAB10\Q2"
PS D:\java\git java\JAVApracticals\LAB10\Q2> javac ProducerConsumer.java
PS D:\java\git java\JAVApracticals\LAB10\Q2> java ProducerConsumer.java
Produced: 1
Produced: 2
Produced: 3
Consumed: 1
Consumed: 2
Consumed: 3
Produced: 4
Produced: 5
Produced: 6
Consumed: 4
Consumed: 5
Consumed: 6
Produced: 7
Produced: 8
Produced: 9
Consumed: 7
Consumed: 8
Consumed: 9
Produced: 10
Consumed: 10
Production and consumption completed.
PS D:\java\git java\JAVApracticals\LAB10\Q2>
```

PROBLEM3

3. (For Advanced Learners – home) Reproduce a deadlock between two threads that lock two objects in opposite order, then remove it with a consistent lock order. Choose the scenario yourself.

CODE

```
public class DeadlockDemo {
   static final Object lockA = new Object();
   static final Object lockB = new Object();

   public static void main(String[] args)
         throws InterruptedException {

      Thread t1 = new Thread(() -> {
         synchronized (lockA) {
            System.out.println("Thread 1 locked A");

            try {
               Thread.sleep(100);
            } catch (InterruptedException e) {
               Thread.currentThread().interrupt();
               return;
            }

            synchronized (lockB) {
               System.out.println("Thread 1 locked B");
            }
         }
      });
```

```
        Thread t2 = new Thread(() -> {
            synchronized (lockB) {
                System.out.println("Thread 2 locked B");

                try {
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }

                synchronized (lockA) {
                    System.out.println("Thread 2 locked A");
                }
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Program completed.");
    }
}
```

OUTPUT

```
Production and consumption completed.
PS D:\java\git java\JAVApracticals\LAB10\Q2> cd "D:\java\git java\JAVApracticals\LAB10\Q3"
PS D:\java\git java\JAVApracticals\LAB10\Q3> javac DeadlockDemo.java
PS D:\java\git java\JAVApracticals\LAB10\Q3> java DeadlockDemo.java
Thread 1 locked A
Thread 2 locked B
```

PRACTICAL - 11

PROBLEM1

1. Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

CODE

```
import java.io.*;

class Student implements Serializable {
    int id;
    String name;
    transient String password;
```

```
    Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }
}

public class SerializationDemo {
    public static void main(String[] args) {
        try {
            Student[] students = {
                new Student(1, "Bhavya", "abc123"),
                new Student(2, "Rahul", "xyz456")
            };

            // Save objects to file
            ObjectOutputStream out =
                new ObjectOutputStream(new FileOutputStream("student.dat"));

            out.writeObject(students);
            out.close();

            // Read objects from file
            ObjectInputStream in =
                new ObjectInputStream(new FileInputStream("student.dat"));

            Student[] loadedStudents = (Student[]) in.readObject();
            in.close();

            // Display loaded objects
            for (Student s : loadedStudents) {
                System.out.println("ID: " + s.id);
                System.out.println("Name: " + s.name);
                System.out.println("Password: " + s.password);
                System.out.println();
            }

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB11\Q1> cd "D:\java\git java\JAVApracticals\LAB11\Q1"
PS D:\java\git java\JAVApracticals\LAB11\Q1> javac SerializationDemo.java
PS D:\java\git java\JAVApracticals\LAB11\Q1> java SerializationDemo
ID: 1
Name: Bhavya
Password: null

ID: 2
Name: Rahul
Password: null

PS D:\java\git java\JAVApracticals\LAB11\Q1>
```

PROBLEM2

2. Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

CODE

```
import java.io.*;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.*;

public class LogAnalyzer {
   public static void main(String[] args) {
      try {
         // Create sample text files
         Files.write(Paths.get("log1.txt"), Arrays.asList(
            "Application started",
            "Error: File not found",
            "User logged in"
         ));

         Files.write(Paths.get("log2.txt"), Arrays.asList(
            "Connection successful",
            "Error: Connection failed",
            "Application closed"
         ));

         int totalLines = 0;
         int keywordCount = 0;
         String keyword = "Error";

         String[] fileNames = {"log1.txt", "log2.txt"};

         for (String fileName : fileNames) {
            Path path = Paths.get(fileName);

            // Read file line by line
            List<String> lines = Files.readAllLines(path);

            totalLines += lines.size();

            for (String line : lines) {
               if (line.contains(keyword)) {
```

```
                keywordCount++;
              }
            }

            // Get file attributes
            BasicFileAttributes attr =
              Files.readAttributes(path, BasicFileAttributes.class);

            System.out.println("File: " + fileName);
            System.out.println("Size: " + attr.size() + " bytes");
            System.out.println();
          }

          System.out.println("Total lines: " + totalLines);
          System.out.println("Lines containing '" + keyword +
                  "': " + keywordCount);

        } catch (IOException e) {
          System.out.println("Error: " + e.getMessage());
        }
      }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB11> cd "D:\java\git java\JAVApracticals\LAB11\Q2"
PS D:\java\git java\JAVApracticals\LAB11\Q2> javac LogAnalyzer.java
PS D:\java\git java\JAVApracticals\LAB11\Q2> java LogAnalyzer.java
File: log1.txt
Size: 60 bytes

File: log2.txt
Size: 69 bytes

Total lines: 6
Lines containing 'Error': 2
PS D:\java\git java\JAVApracticals\LAB11\Q2>
```

PROBLEM3

3. (For Advanced Learners – home) Walk a folder tree with NIO and produce a report of the files it contains (name, size, modified time), writing the summary to a file; decide what to include.

CODE

```
import java.io.*;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.Instant;

public class FolderTreeReport
{
  public static void main(String[] args)
  {
    Path folder = Paths.get("sample");
    Path report = Paths.get("report.txt");

    try
```

```
    {
      // Create a sample folder and files
      Files.createDirectories(folder.resolve("subfolder"));

      Files.write(folder.resolve("file1.txt"),"Hello Java".getBytes());

      Files.write(folder.resolve("subfolder/file2.txt"),"Java NIO".getBytes());

      try (BufferedWriter writer =Files.newBufferedWriter(report))
      {

        writer.write("FOLDER TREE REPORT");
        writer.newLine();
        writer.write("------------------");
        writer.newLine();

        Files.walkFileTree(folder,new SimpleFileVisitor<Path>()
          {

          @Override
          public FileVisitResult visitFile(Path file, BasicFileAttributes attrs)

          throws IOException
          {

            writer.write("Name: " +folder.relativize(file));
            writer.newLine();

            writer.write("Size: " + attrs.size() + " bytes");
            writer.newLine();

            writer.write("Modified: " + Instant.ofEpochMilli(attrs.lastModifiedTime().toMillis()));
            writer.newLine();

            writer.newLine();

            return FileVisitResult.CONTINUE;
          }
        }
      );
      }

      System.out.println("Report created successfully!");
      System.out.println("Report file: report.txt");

    }
    catch (IOException e)
    {
      System.out.println("Error: " + e.getMessage());
    }
  }
```

	}

```
        {
            return map.get(b).compareTo(map.get(a));
        }
    }
  );

    System.out.println("Word Frequency:");

    for (String word : list)
    {
        System.out.println(word + " : " + map.get(word));
    }
  }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB12\Q1> cd "D:\java\git java\JAVApracticals\LAB12\Q1"
PS D:\java\git java\JAVApracticals\LAB12\Q1> javac WordFrequency.java
PS D:\java\git java\JAVApracticals\LAB12\Q1> java WordFrequency.java
Word Frequency:
java : 2
is : 2
powerful : 1
and : 1
easy : 1
PS D:\java\git java\JAVApracticals\LAB12\Q1>
```

PROBLEM2

2. Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

CODE

```
import java.util.*;

public class UniqueVisitors
{
  public static void main(String[] args)
  {

    String[] logins = {
      "Bhavya", "Rahul", "Jay", "Bhavya",
      "Het", "Rahul", "Fenil"
    };

    HashSet<String> uniqueUsers = new HashSet<>();
    LinkedHashSet<String> orderedUsers = new LinkedHashSet<>();

    for (String name : logins) {uniqueUsers.add(name);orderedUsers.add(name);}

    System.out.println("Unique users count: " + uniqueUsers.size());
    System.out.println("First three distinct users:");

    int count = 0;
```

```
        for (String name : orderedUsers)
        {
            if (count == 3)
            {
                break;
            }

            System.out.println(name);
            count++;
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB12> cd "D:\java\git java\JAVApracticals\LAB12\Q2"
PS D:\java\git java\JAVApracticals\LAB12\Q2> javac UniqueVisitors.java
PS D:\java\git java\JAVApracticals\LAB12\Q2> java UniqueVisitors.java
Unique users count: 5
First three distinct users:
Bhavya
Rahul
Jay
PS D:\java\git java\JAVApracticals\LAB12\Q2>
```

PROBLEM3

3. (For Advanced Learners – home) Build a leaderboard from player scores held in a Map and print the top-N sorted by score (ties broken by name); choose the data and sorting details.

CODE

```
import java.util.*;

public class Leaderboard
{
    public static void main(String[] args)
    {

        Map<String, Integer> scores = new HashMap<>();

        scores.put("Bhavya", 95);
        scores.put("Rahul", 88);
        scores.put("Jay", 95);
        scores.put("Het", 78);
        scores.put("Fenil", 88);

        List<Map.Entry<String, Integer>> players =new ArrayList<>(scores.entrySet());

        Collections.sort(players, new Comparator<Map.Entry<String, Integer>>()
        {
            public int compare(Map.Entry<String, Integer> p1,Map.Entry<String, Integer> p2)
            {
                int result = p2.getValue().compareTo(p1.getValue());

                if (result == 0)
                {
                    return p1.getKey().compareTo(p2.getKey());
```

```
            }

            return result;
        }
    }
);

    int topN = 3;

    System.out.println("Top " + topN + " Players:");
    System.out.println("------------------");

    for (int i = 0; i < Math.min(topN, players.size()); i++)
    {
        Map.Entry<String, Integer> player = players.get(i);

        System.out.println(player.getKey() + " - " + player.getValue());
    }
  }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB12> cd "D:\java\git java\JAVApracticals\LAB12\Q3"
PS D:\java\git java\JAVApracticals\LAB12\Q3> javac Leaderboard.java
PS D:\java\git java\JAVApracticals\LAB12\Q3> java Leaderboard.java
Top 3 Players:
------------------
Bhavya - 95
Jay - 95
Fenil - 88
PS D:\java\git java\JAVApracticals\LAB12\Q3>
```

PRACTICAL - 13

PROBLEM1	Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

CODE	class Box<T>
	{
	  private T value;
	
	  void set(T value)
	  {
	  this.value = value;
	  }
	
	  T get()

```
    {
        return value;
    }
}

class Pair<A, B>
{
    private A first;
    private B second;

    Pair(A first, B second)
    {
        this.first = first;
        this.second = second;
    }

    A getFirst()
    {
        return first;
    }

    B getSecond()
    {
        return second;
    }
}

public class ContainerDemo
{

    static <T> T first(T[] values)
    {
        if (values.length == 0)
        {
            throw new IllegalArgumentException("Empty array");
        }
        return values[0];
    }

    public static void main(String[] args)
    {

        Box<String> names = new Box<>();
        names.set("Bhavya");

        Box<Integer> marks = new Box<>();
        marks.set(90);

        Pair<String, Integer> pair =new Pair<>(names.get(), marks.get());

        System.out.println(pair.getFirst() + " " + pair.getSecond());
```

```
        System.out.println(first(new String[]{"Java", "Python"}));

        System.out.println(first(new Integer[]{10, 20}));
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB13\Q1> cd "D:\java\git java\JAVApracticals\LAB13\Q1"
PS D:\java\git java\JAVApracticals\LAB13\Q1> javac ContainerDemo.java
PS D:\java\git java\JAVApracticals\LAB13\Q1> java ContainerDemo.java
Bhavya 90
Java
10
PS D:\java\git java\JAVApracticals\LAB13\Q1>
```

PROBLEM2

Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

CODE

```
public class Stats
{

    static <N extends Number> double average(N[] values)
    {
        if (values.length == 0)
        {
            throw new IllegalArgumentException("Empty array");
        }

        double sum = 0;

        for (N n : values)
        {
            sum += n.doubleValue();
        }

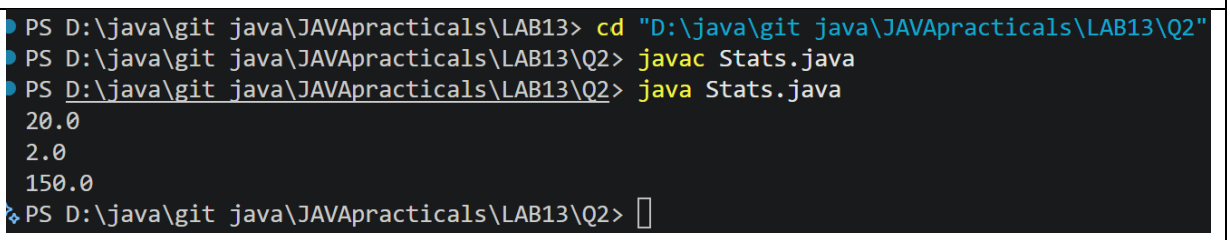
        return sum / values.length;
    }

    public static void main(String[] args)
    {

        System.out.println(average(new Integer[]{10, 20, 30}));

        System.out.println(average(new Double[]{1.5, 2.5}));

        System.out.println(average(new Long[]{100L, 200L}));
    }
}
```

OUTPUT	
PROBLEM3	(For Advanced Learners – home) Implement a small generic container of your choice (e.g. a stack) backed by an array, reuse it for two different types, and handle the empty case sensibly.

CODE	import java.util.EmptyStackException;
	
	class Stack<T>
	{
	private Object[] data;
	private int size;
	
	Stack(int capacity)
	{
	data = new Object[capacity];
	}
	
	void push(T item)
	{
	if (size == data.length)
	{
	throw new IllegalStateException("Stack full");
	}
	
	data[size++] = item;
	}
	
	@SuppressWarnings("unchecked")
	T pop()
	{
	if (size == 0)
	{
	throw new EmptyStackException();
	}

```
        T item = (T) data[--size];
        data[size] = null;

        return item;
    }
}

public class StackDemo
{
    public static void main(String[] args)
    {

        Stack<Integer> numbers = new Stack<>(3);
        numbers.push(10);
        numbers.push(20);

        Stack<String> names = new Stack<>(2);
        names.push("Bhavya");

        System.out.println(numbers.pop());
        System.out.println(names.pop());

        try
        {
            names.pop();
        }
        catch (EmptyStackException e)
        {
            System.out.println("Stack empty");
        }
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB13> cd "D:\java\git java\JAVApracticals\LAB13\Q3"
PS D:\java\git java\JAVApracticals\LAB13\Q3> javac StackDemo.java
PS D:\java\git java\JAVApracticals\LAB13\Q3> java StackDemo.java
20
Bhavya
Stack empty
PS D:\java\git java\JAVApracticals\LAB13\Q3>
```

PRACTICAL – 14

1) Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

Item.java

CODE

```
public class Item {
    private String name;

    public Item(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

ItemService.java

CODE

```
import java.util.ArrayList;

public class ItemService {
    private ArrayList<Item> items = new ArrayList<>();

    public void add(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name required");
        }

        items.add(new Item(name));
    }

    public int count() {
        return items.size();
    }

    public ArrayList<Item> all() {
        return new ArrayList<>(items);
    }

    public void invalidAction() {
        throw new IllegalArgumentException("Invalid action");
    }
}
```

ItemApp.java

CODE

```
import java.util.Scanner;

public class ItemApp {
    public static void main(String[] args) {
        ItemService service = new ItemService();
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n1. Add");
```

```
            System.out.println("2. List");
            System.out.println("3. Exit");
            System.out.print("Enter choice: ");

            String choice = sc.nextLine();

            try {
                switch (choice) {
                    case "1":
                        System.out.print("Enter item name: ");
                        service.add(sc.nextLine());
                        System.out.println("Item added successfully");
                        break;

                    case "2":
                        for (Item item : service.all()) {
                            System.out.println(item.getName());
                        }
                        break;

                    case "3":
                        System.out.println("Exiting...");
                        sc.close();
                        return;

                    default:
                        service.invalidAction();
                }
            } catch (IllegalArgumentException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

ItemServiceTest.java

CODE

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class ItemServiceTest {

    @Test
    void addIncreasesCount() {
        ItemService service = new ItemService();

        service.add("Book");

        assertEquals(1, service.count());
    }

    @Test
    void blankNameThrowsException() {
        ItemService service = new ItemService();

        assertThrows(
```

```
            IllegalArgumentException.class,
            () -> service.add("")
        );
    }

    @Test
    void invalidActionThrowsException() {
        ItemService service = new ItemService();

        assertThrows(
            IllegalArgumentException.class,
            () -> service.invalidAction()
        );
    }
}
```

OUTPUT

```
PS D:\java\git java\JAVApracticals\LAB14> cd "D:\java\git java\JAVApracticals\LAB1
PS D:\java\git java\JAVApracticals\LAB14\Q1> javac Item.java ItemService.java Item
PS D:\java\git java\JAVApracticals\LAB14\Q1> java ItemApp

 1. Add
 2. List
 3. Exit
 Enter choice: 1
 Enter item name: pen
 Item added successfully

 1. Add
 2. List
 3. Exit
 Enter choice: 2
 pen

 1. Add
 2. List
 3. Exit
 Enter choice: 4
 Invalid action

 1. Add
 2. List
 3. Exit
 Enter choice: 3
 Exiting...
PS D:\java\git java\JAVApracticals\LAB14\Q1>
```

2) (For Advanced Learners – home) Organise that app into packages, build a runnable JAR,
and add a test that saves and reloads its data unchanged.

Sec/model/Item.java

Code

```
package app.model;

import java.io.Serializable;

public class Item implements Serializable {
    private static final long serialVersionUID = 1L;

    private String name;

    public Item(String name) {
        this.name = name;
```

```
    }

    public String getName() {
        return name;
    }

    @Override
    public String toString() {
        return name;
    }
}
```

Src/service/ItemService.java

Code

```
package app.service;

import app.model.Item;
import java.io.*;
import java.nio.file.*;
import java.util.ArrayList;
import java.util.List;

public class ItemService {
    private ArrayList<Item> items = new ArrayList<>();

    public void add(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name required");
        }

        items.add(new Item(name));
    }

    public int count() {
        return items.size();
    }

    public ArrayList<Item> all() {
        return new ArrayList<>(items);
    }

    public void save(Path file) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(
                Files.newOutputStream(file))) {
            out.writeObject(items);
        }
    }

    @SuppressWarnings("unchecked")
    public void load(Path file)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(
                Files.newInputStream(file))) {

            Object data = in.readObject();

            if (!(data instanceof ArrayList<?>)) {
                throw new IOException("Invalid data file");
            }

            ArrayList<?> loaded = (ArrayList<?>) data;

            for (Object obj : loaded) {
                if (!(obj instanceof Item)) {
                    throw new IOException("Invalid item data");
                }
            }
```

```
            items = new ArrayList<>((List<Item>) loaded);
        }
    }
}
```

Src/app/ItemApp.java

Code

```
package app.main;

import app.model.Item;
import app.service.ItemService;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.Scanner;

public class ItemApp {
    public static void main(String[] args) {
        ItemService service = new ItemService();
        Path file = Paths.get("items.dat");

        try {
            if (java.nio.file.Files.exists(file)) {
                service.load(file);
                System.out.println("Data loaded successfully.");
            }
        } catch (Exception e) {
            System.out.println("Could not load data: " + e.getMessage());
        }

        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1. Add item");
                System.out.println("2. List items");
                System.out.println("3. Save data");
                System.out.println("4. Exit");
                System.out.print("Enter choice: ");

                String choice = sc.nextLine();

                try {
                    switch (choice) {
                        case "1":
                            System.out.print("Enter item name: ");
                            service.add(sc.nextLine());
                            System.out.println("Item added.");
                            break;

                        case "2":
                            for (Item item : service.all()) {
                                System.out.println(item.getName());
                            }
                            break;

                        case "3":
                            service.save(file);
                            System.out.println("Data saved successfully.");
                            break;

                        case "4":
                            System.out.println("Goodbye!");
                            return;

                        default:
                            System.out.println("Invalid choice.");
                    }
                } catch (Exception e) {
```

```
                System.out.println("Error: " + e.getMessage());
            }
        }
    }
}
```

test/ItemServiceTest.java

Code

```
import app.model.Item;
import app.service.ItemService;
import org.junit.jupiter.api.Test;

import java.nio.file.Files;
import java.nio.file.Path;

import static org.junit.jupiter.api.Assertions.*;

public class ItemServiceTest {

    @Test
    void saveAndLoadPreservesData() throws Exception {
        Path file = Files.createTempFile("items", ".dat");

        try {
            ItemService original = new ItemService();
            original.add("Book");
            original.add("Pen");

            original.save(file);

            ItemService restored = new ItemService();
            restored.load(file);

            assertEquals(original.count(), restored.count());
            assertEquals("Book", restored.all().get(0).getName());
            assertEquals("Pen", restored.all().get(1).getName());
        } finally {
            Files.deleteIfExists(file);
        }
    }

    @Test
    void addIncreasesCount() {
        ItemService service = new ItemService();

        service.add("Book");

        assertEquals(1, service.count());
    }

    @Test
    void blankNameThrowsException() {
        ItemService service = new ItemService();

        assertThrows(
            IllegalArgumentException.class,
            () -> service.add(" ")
        );
    }
}
```

Output

```
PS D:\java\git java\JAVApracticals\LAB14> cd "D:\java\git java\JAVApracticals\LAB14\Q2"
PS D:\java\git java\JAVApracticals\LAB14\Q2> javac -d out src/app/model/Item.java src/app/service/ItemService.java src/app/main/ItemAp
p.java
PS D:\java\git java\JAVApracticals\LAB14\Q2> java -cp out app.main.ItemApp

1. Add item
2. List items
3. Save data
4. Exit
Enter choice: 1
Enter item name: book
Item added.

1. Add item
2. List items
3. Save data
4. Exit
Enter choice: 2
book

1. Add item
2. List items
3. Save data
4. Exit
Enter choice: 3
Data saved successfully.

1. Add item
2. List items
3. Save data
4. Exit
Enter choice: 5
Invalid choice.

1. Add item
2. List items
3. Save data
4. Exit
Enter choice: 4
```